

24DIT041

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology and Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Department of Information Technology

Big Data Analytics (CSUE301)

24DIT041

1. Aim

To implement a Hadoop MapReduce program to find the maximum temperature recorded for each year from NCDC weather data and identify the hottest year in the dataset.

2. Problem Definition

A meteorological research organization maintains historical weather records collected from different weather stations. The NCDC weather dataset contains temperature records for different years.

The objective is to process the weather data using Hadoop MapReduce, extract the year and temperature values, calculate the maximum temperature for each year, and identify the year with the highest recorded temperature.

3. Objectives

- To upload the NCDC weather dataset to HDFS.
 - To understand the structure of weather records.
 - To implement a Mapper to extract year and temperature.
 - To implement a Reducer to find the maximum temperature for each year.
 - To execute the MapReduce job using Hadoop.
 - To store and verify the results in HDFS.
 - To identify the hottest year in the dataset.
 - To understand the advantages of MapReduce for large-scale weather data analysis.
-

4. Software and Technologies Used

- Hadoop 3.3.6
 - HDFS
 - MapReduce
 - Java
 - JDK
 - Ubuntu/Linux
 - Terminal
-

5. Dataset

The NCDC weather dataset used for this practical contains the year, weather station/country information, and recorded temperature.

Input Dataset

24DIT041

2019,US,35

2019,IN,41

2019,UK,29

2020,US,38

2020,IN,44

2020,UK,31

2021,US,40

2021,IN,46

2021,UK,33

2022,US,42

2022,IN,48

2022,UK,35

2023,US,45

2023,IN,50

2023,UK,37

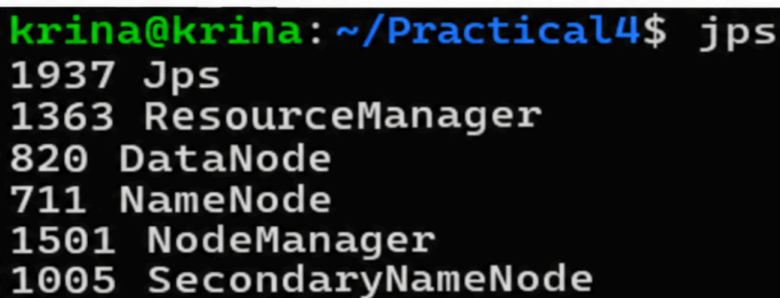
```
krina@krina: ~/Practical4$ cat input.txt
2019,US,35
2019,IN,41
2019,UK,29
2020,US,38
2020,IN,44
2020,UK,31
2021,US,40
2021,IN,46
2021,UK,33
2022,US,42
2022,IN,48
2022,UK,35
2023,US,45
2023,IN,50
2023,UK,37
```

6. Hadoop Environment Configuration

The Hadoop environment was started using HDFS and YARN services. The running Hadoop services were verified using the `jps` command.

The main services include:

- NameNode
- DataNode
- SecondaryNameNode
- ResourceManager
- NodeManager



```
krina@krina: ~/Practical4$ jps
1937 Jps
1363 ResourceManager
820 DataNode
711 NameNode
1501 NodeManager
1005 SecondaryNameNode
```

7. Mapper Program

The Mapper reads each weather record, separates the fields using a comma, extracts the year and temperature, and generates an intermediate key-value pair in the form:

(Year, Temperature)

MaxTempMapper.java import

java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

public class MaxTempMapper extends Mapper<Object, Text, Text, IntWritable> { private

Text year = new Text();

private IntWritable temperature = new IntWritable(); public

void map(Object key, Text value, Context context)

throws IOException, InterruptedException

{ String[] fields = value.toString().split(",");

if (fields.length >= 3) { year.set(fields[0].trim());

temperature.set(Integer.parseInt(fields[2].trim()));

context.write(year, temperature);

}

}

```
}
```

```
kria@krina:~/Practical4$ cat MaxTempMapper.java
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class MaxTempMapper extends Mapper<Object, Text, Text, IntWritable> {

    private Text year = new Text();
    private IntWritable temperature = new IntWritable();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String[] fields = value.toString().split(",");

        if (fields.length >= 3) {
            year.set(fields[0].trim());
            temperature.set(Integer.parseInt(fields[2].trim()));

            context.write(year, temperature);
        }
    }
}
```

8. Reducer Program

The Reducer receives all temperature values associated with the same year and compares them to find the maximum temperature.

MaxTempReducer.java import

```
java.io.IOException;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

```
public class MaxTempReducer extends Reducer<Text, IntWritable, Text, IntWritable>
```

```
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
```

```
        throws IOException, InterruptedException
```

```
    { int maxTemperature = Integer.MIN_VALUE;
```

```
      for (IntWritable value : values) {
```

```
          maxTemperature = Math.max(maxTemperature, value.get());
```

```
      }
```

```
      context.write(key, new IntWritable(maxTemperature));
```

```
    }
```

```
}
```

```
krina@krina:~/Practical4$ cat MaxTemperature.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxTemperature {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Maximum Temperature");

        job.setJarByClass(MaxTemperature.class);
        job.setMapperClass(MaxTempMapper.class);
        job.setReducerClass(MaxTempReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }
}
```

9. Driver Program

The Driver program configures the MapReduce job, specifies the Mapper and Reducer classes, defines the input and output paths, and executes the job.

MaxTemperature.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MaxTemperature {
    public static void main(String[] args) throws Exception
    {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Maximum Temperature");
        job.setJarByClass(MaxTemperature.class);
        job.setMapperClass(MaxTempMapper.class);
        job.setReducerClass(MaxTempReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
```

```

        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

```

krina@krina:~/Practical4$ jar tf maxTemperature.jar maxTemperature.jar
total 32K
-rw-r--r--  1 kr6  kr6  1.8K Sep 13 18:51 MaxTempMapper.class
-rw-r--r--  1 kr6  kr6   696 Sep 13 18:46 MaxTempMapper.java
-rw-r--r--  1 kr6  kr6  1.7K Sep 13 18:51 MaxTempReducer.class
-rw-r--r--  1 kr6  kr6   609 Sep 13 18:47 MaxTempReducer.java
-rw-r--r--  1 kr6  kr6  1.4K Sep 13 18:51 MaxTemperature.class
-rw-r--r--  1 kr6  kr6  2.9K Sep 13 18:51 MaxTemperature.jar
-rw-r--r--  1 kr6  kr6   970 Sep 13 18:50 MaxTemperature.java
-rw-r--r--  1 kr6  kr6   165 Sep 13 18:45 input.txt

```

10. Compilation and JAR Creation

The Java programs were compiled using Hadoop's Java compiler:

```
hadoop com.sun.tools.javac.Main MaxTempMapper.java MaxTempReducer.java
MaxTemperature.java
```

A JAR file was then created:

```
jar cf MaxTemperature.jar MaxTempMapper*.class MaxTempReducer*.class
MaxTemperature.class
```

The generated JAR file was verified using the ls command.

```

krina@krina:~/Practical4$ hdfs dfs -cat /user/kr6/practical4/input/input.txt
2026-09-13 18:52:29,606 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.... using builtin-java classes where applicable
applicable
2019,IN,41
2019,UK,29
2020,US,38
2020,IN,44
2020,UK,31
2021,US,40
2021,IN,46
2022,US,42
2022,IN,48
2022,UK,35
2023,US,45
2023,IN,50
2023,UK,37

```

11. Upload Dataset to HDFS

The HDFS input directory was created using:

```
hdfs dfs -mkdir -p /user/kr6/practical4/input
```

The dataset was uploaded using:

```
hdfs dfs -put -f input.txt /user/kr6/practical4/
```

input/ The uploaded dataset was verified using:

```
hdfs dfs -ls /user/kr6/practical4/input
```

and:

hdfs dfs -cat /user/kr6/practical4/input/input.txt

```
krina@krina:~/Practical4$ hadoop jar /home/krina/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount /input/output
2026-09-13 18:53:05,796 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-13 18:53:06,442 INFO client.RMProxy: Running job: job_1789324963986_0001
2026-09-13 18:53:07,319 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-13 18:53:07,412 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-krina/staging/kr6/staging/job_1789324963986_0001
2026-09-13 18:53:07,787 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 18:53:08,106 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 18:53:08,168 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789324963986_0001
2026-09-13 18:53:08,169 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 18:53:08,941 INFO conf.Configuration: resource-types.xml not found
2026-09-13 18:53:09,314 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 18:53:09,421 INFO impl.YarnClientImpl: Submitted application application_1789324963986_0001
2026-09-13 18:53:09,436 WARN mapreduce.Job: The url to track the job: http://krina:8088/proxy/application_1789324963986_0001/
2026-09-13 18:53:09,732 INFO mapreduce.Job: Running job: job_1789324963986_0001
2026-09-13 18:53:10,655 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 18:53:10,727 INFO mapreduce.Job: Job job_1789324963986_0001 running in uber mode : false
2026-09-13 18:53:16,716 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 18:53:16,748 INFO mapreduce.Job: Job job_1789324963986_0001
2026-09-13 18:53:17,748 INFO mapreduce.Job: Job job_1789324963986_0001 completed successfully
2026-09-13 18:53:17,748 INFO mapreduce.Job: Counters: 54

File System Counters
  FILE: Number of bytes read=171
  FILE: Number of bytes written=552367
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=287
  HDFS: Number of bytes written=9
  HDFS: Number of read operations=0
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=0
  HDFS: Number of bytes read erasure-coded=0

Job Counters
  Launched map tasks=1
  Launched reduce tasks=1
  Data-local map tasks=1
  Total time spent by all maps in occupied slots (ms)=4881
  Total time spent by all reduces in occupied slots (ms)=3513
  Total time spent by all map tasks (ms)=4481
  Total time spent by all reduce tasks (ms)=3513
  Total vcore-milliseconds taken by all map tasks=4481
  Total vcore-milliseconds taken by all reduce tasks=3513
  Total megabyte-milliseconds taken by all map tasks=4589584
  Total megabyte-milliseconds taken by all reduce tasks=3597312

Map-Reduce Framework
  Map input records=15
  Map output records=15
  Map output bytes=138
  Map output materialized bytes=171
  Input split bytes=122
  Combine input records=0
  Combine output records=0
  Reduce input groups=5
  Reduce shuffle bytes=171
  Reduce input records=15
  Reduce output records=5
  Spilled Records=30
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=52
  CPU time spent (ms)=1120
  Physical memory (bytes) snapshot=458831952
  Virtual memory (bytes) snapshot=5470994432
  Total committed heap usage (bytes)=4259696976
  Peak Map Physical memory (bytes)=2612893952
  Peak Map Virtual memory (bytes)=2728599552
  Peak Reduce Physical memory (bytes)=1895260416
  Peak Reduce Virtual memory (bytes)=2742394880

Shuffle Error
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  File Input Format Counters
    Bytes Read=171
  File Output Format Counters
    Bytes Written=9
```

12. MapReduce Execution

Before executing the job, the previous output directory was removed: hdfs

dfs -rm -r -f /user/kr6/practical4/output

The MapReduce job was executed using:

hadoop jar MaxTemperature.jar MaxTemperature /user/kr6/practical4/input
/user/kr6/practical4/output

The job completed successfully with Map and Reduce progress reaching 100%.

```
krina@krina:~/Practical4$ hdfs dfs -cat /user/kr6/practical4/output/part-r-000000
2026-09-13 19:20:45,938 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2019      41
2020      44
2021      46
2022      48
2023      50
```

13. Yearly Maximum Temperature Output

The final output was displayed using:

hdfs dfs -cat /user/kr6/practical4/output/part-r-000000

Output

```
krina@krina:~/Practical4$ hdfs dfs -ls /user/kr6/practical4/output
2026-09-13 18:56:10,381 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 kr6 supergroup 0 2026-09-13 18:53 /user/kr6/practical4/output/_SUCCESS
-rw-r--r-- 1 kr6 supergroup 48 2026-09-13 18:53 /user/kr6/practical4/output/part-r-000000
krina@krina:~/Practical4$ hdfs dfs -cat /user/kr6/practical4/output/part-r-000000
2026-09-13 18:56:16,193 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2019      41
2020      44
2021      46
2022      48
2023      50
```

14. Hottest Year

From the MapReduce output, the maximum temperature recorded for each year was compared.

The highest temperature was:

Year	Maximum Temperature
2019	41°C
2020	44°C
2021	46°C
2022	48°C
2023	50°C

Year: 2023

Maximum Temperature: 50°C

Therefore, 2023 was the hottest year in the given dataset.

```
krina@krina:~/Practical4$ hdfs dfs -cat /user/kr6/practical4/output/part-r-00000
2026-09-13 18:54:42,994 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2019    41
2020    44
2021    46
2022    48
2023    50
```

15. Key Questions / Analysis

Q1. How is weather data parsed and processed using MapReduce?

The Mapper reads each weather record and splits it into fields. It extracts the year and temperature and generates a key-value pair in the form (Year, Temperature). The Reducer receives all temperatures belonging to the same year and finds the maximum value.

Q2. What key-value pairs are generated by the Mapper? The

Mapper generates:

Key = Year

Value = Temperature For

example:

2019	35
2019	41
2019	29

Q3. How does the Reducer identify the maximum temperature for a given year?

The Reducer receives all temperature values for the same year. It compares the values one by one

and stores the highest value. Finally, it outputs the year and its maximum temperature.

Q4. What challenges arise when processing large climate datasets?

Large climate datasets may contain huge numbers of records, missing or invalid values, different data formats, network communication overhead, and high storage requirements. Processing such data on a single machine can also take a long time.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop distributes large datasets across multiple machines using HDFS and processes them in parallel using MapReduce. This reduces processing time, provides fault tolerance, and allows the system to handle datasets much larger than a single machine can efficiently process.

16. Supplementary Problems

A. Find the minimum temperature recorded for each year

From the given dataset:

Year	Minimum Temperature
2019	29°C
2020	31°C
2021	33°C
2022	35°C
2023	37°C

B. Calculate average yearly temperatures

Year	Average Temperature
2019	35.00°C
2020	37.67°C
2021	39.67°C
2022	41.67°C
2023	44.00°C

C. Identify the year with the highest recorded temperature overall The

highest recorded temperature is 50°C, which occurred in 2023.

17. Final HDFS Output Verification The

output directory was verified using:

```
hdfs dfs -ls /user/kr6/practical4/output
```

The final output was verified using:

```
hdfs dfs -cat /user/kr6/practical4/output/part-r-00000
```

The verified output was:

```
2019  41
2020  44
2021  46
      1
2022  48
      2
2023  50
      3
```

18. Result

The Hadoop MapReduce program was successfully implemented to find the maximum temperature recorded for each year from the NCDC weather dataset.

The yearly maximum temperatures were successfully obtained:

- 2019 → 41°C
- 2020 → 44°C
- 2021 → 46°C
- 2022 → 48°C
- 2023 → 50°C

The hottest year was 2023, with a maximum recorded temperature of 50°C.

19. Conclusion

The practical successfully demonstrated the use of Hadoop MapReduce for processing weather data. The Mapper extracted the year and temperature values, while the Reducer calculated the maximum temperature for each year. Hadoop HDFS provided distributed storage and MapReduce enabled efficient processing of large datasets. The experiment showed how MapReduce can be used for large-scale climate and weather data analytics.